DSA5205 Project 1: Machine Learning in Finance

xxx
A0123456R

October 2026

Abstract

This report studies how model complexity and regularization affect return prediction. It covers ridge and Lasso simulations, predictions for three hidden test sets, and a small neural-network experiment on overfitting.

1 Task 1: Misspecified Simulation with Ridge

1.1 Design and implementation

Following Kelly, Malamud, and Zhou [1], returns are generated as

$$R_{t+1} = S_t'\beta^* + \varepsilon_{t+1}, \qquad S_t \sim \mathcal{N}(0, I_P), \qquad \varepsilon_{t+1} \sim \mathcal{N}(0,1). \tag{1}$$

The vector β^* is normalized so that $\|\beta^*\|_2^2 = 0.2$, and true complexity is fixed at $c = P/T_{\text{tr}} = 10$. For each replication, the features are randomly ordered once. A fraction q is then revealed, giving observed complexity $cq = P_1/T_{\text{tr}}$. Because some true features are hidden when $q < 1$, the fitted model is misspecified.

The simulation parameters are listed in Table 1 and configured in `task1_2/task1.yaml`. Data generation is implemented by `generate_data()` in `task1_2/generate_data.py`; the experiment loop and nested feature selection are in `task1_2/task1.py`.

Table 1: Task 1 simulation parameters.

Parameter	Value
Base seed and replications	5205; 50
Training and test sizes	$T_{\text{tr}} = 500$; $T_{\text{te}} = 5000$
DGP complexity and dimension	$c = 10$; $P = 5000$
Signal strength and noise	$b^* = 0.2$; $\sigma_\varepsilon = 1$
cq grid	$\{0.50, 0.75, 0.90, 0.95, 0.98, 1.02, 1.05, 1.10,$ $1.25, 1.50, 2, 3, 5, 7, 10\}$
Ridge grid	$z \in \{0, 0.01, 0.1, 1, 10, 50\}$

For each observed block, ridge is estimated without an intercept:

$$\widehat{\beta}(z) = \left(X'X + zT_{\text{tr}}I\right)^{-1} X'y. \tag{2}$$

At $z = 0$, the code uses the pseudoinverse. For $z > 0$, it uses the smaller of the primal and dual systems; see `fit_ridge()` in `task1_2/fit_ridge.py`.

All quantities are evaluated on the independent test set. With $\widehat{R}_{t+1} = S_t'\widehat{\beta}$ and timing return $R^{\pi}_{t+1} = \widehat{R}_{t+1}R_{t+1}$, the reported metrics are

$$R^2_{\text{paper}} = 1 - \frac{\mathbb{E}[(R_{t+1} - \widehat{R}_{t+1})^2]}{\mathbb{E}[R^2_{t+1}]}, \tag{3}$$

$$\mathbb{E}[R^{\pi}_{t+1}] = \mathbb{E}[\widehat{R}_{t+1}R_{t+1}], \tag{4}$$

$$\text{SR} = \frac{\mathbb{E}[R^{\pi}_{t+1}]}{\sqrt{\mathbb{E}[(R^{\pi}_{t+1})^2]}}, \qquad \|\widehat{\beta}(z)\|_2. \tag{5}$$

Their sample analogues are computed by `calculate_metrics()` in `task1_2/test_metrics.py`.

1.2 Results

Figure 1 averages 50 replications. Without shrinkage, the model is unstable near $cq = 1$: R^2_{paper} falls to about -50 and $\|\widehat{\beta}\|_2$ exceeds 7.5. Both improve once $cq > 1$. Ridge shrinkage removes most of this spike; $z = 10$ or 50 keeps the coefficient norm small and R^2_{paper} near zero.

Expected return rises as more features are observed. Sharpe first rises, drops near $cq = 1$, and then rises again. This is the paper's "double ascent" pattern. With stronger shrinkage, the dip is much smaller. At $cq = 10$, Sharpe is about 0.049–0.050, with a maximum of 0.0505 for $z = 50$. For $\Psi = I$, the paper's optimal shrinkage is

$$z^*(q) = \frac{c\{1 + b^*(1-q)\}}{b^*} = 60 - 10q, \tag{6}$$

so $z = 50$ is optimal at $q = 1$. Overall, the simulation supports the paper's main result: with suitable shrinkage, observing more signals improves the timing strategy even when test R^2 is close to zero or negative.

2 Task 2: Alternative Model

2.1 Method and experimental control

Task 2 uses the same data, splits, feature order, and cq grid as Task 1, but replaces Ridge with Lasso:

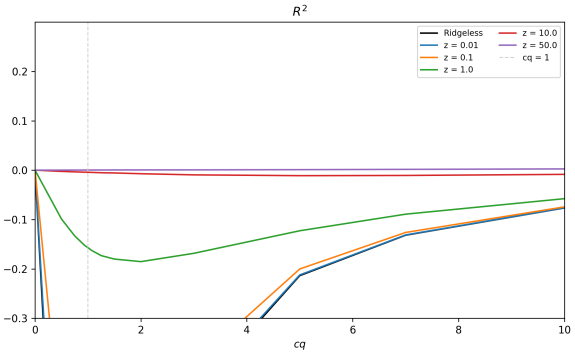
$$\widehat{\beta}(\alpha) = \arg\min_{\beta} \left\{ \frac{1}{2T_{\text{tr}}} \|y - X\beta\|_2^2 + \alpha\|\beta\|_1 \right\}, \tag{7}$$

The grid is $\alpha \in \{0.001, 0.003, 0.01, 0.03, 0.1\}$. The code is in `task1_2/task2.py`, with settings in `task1_2/task2.yaml`. The same test metrics are used, so the comparison with Ridge is direct.

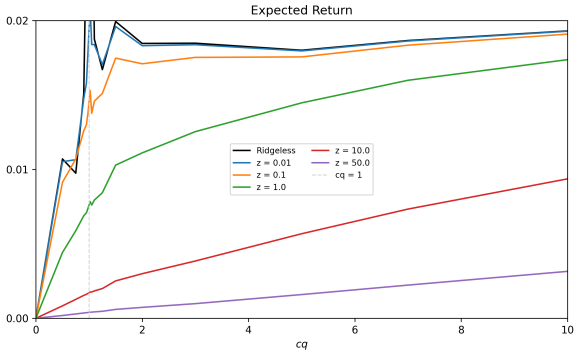
2.2 Results and comparison with ridge

Figure 2 again averages 50 replications. Small penalties still produce instability near $cq = 1$, although it is less extreme than ridgeless regression. Larger penalties reduce the coefficient norm, but $\alpha = 0.1$ shrinks the model too much and gives low return and Sharpe.

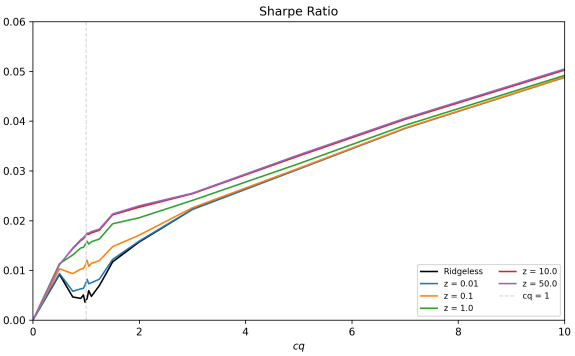
Lasso still improves as more features are revealed, but it performs worse than the best Ridge model. At $cq = 10$, its best Sharpe is 0.0224, compared with 0.0505 for Ridge. This is expected because the true signal is dense: many features are individually weak but useful together. Lasso removes too many of them, while Ridge keeps them with moderate shrinkage.



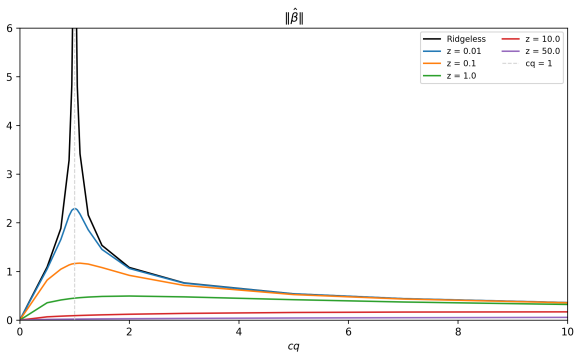
(a) R^2_{paper}.



(b) Expected timing return.

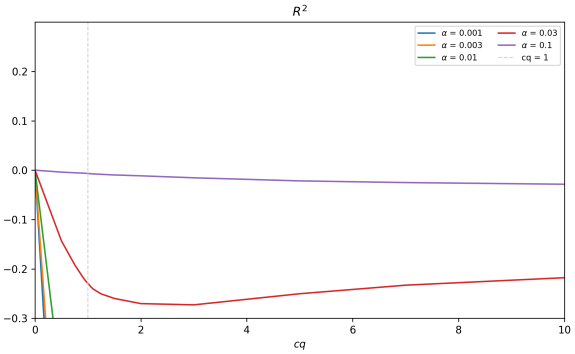


(c) Paper Sharpe ratio.

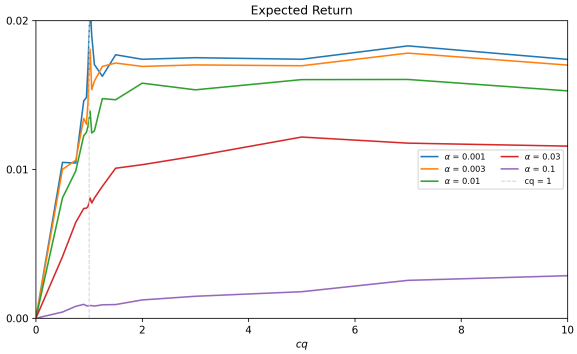


(d) Coefficient norm.

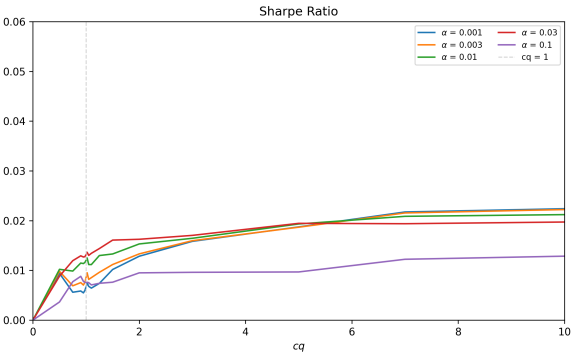
Figure 1: Task 1 results averaged over 50 replications. The dashed line marks $cq = 1$; the axes clip some extreme values near interpolation.



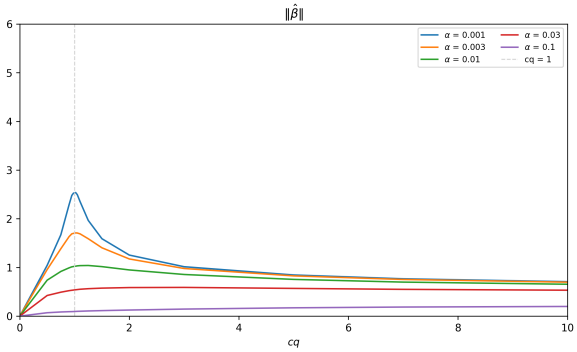
(a) R^2_{paper}.



(b) Expected timing return.



(c) Paper Sharpe ratio.



(d) Coefficient norm.

Figure 2: Task 2 Lasso results averaged over 50 replications. The dashed line marks $cq = 1$; the axes clip some extreme values.

3 Task 3: Predictions for Data Sets A, B, and C

3.1 Validation and model selection

The time column is not used as a predictor. The last 20% of each training file is used for validation, giving 120 observations for A, 48 for B, and 72 for C. Ridge, Lasso, and a one-hidden-layer neural network are compared using R^2_{paper}. Return and Sharpe are also reported, but they are not used to change the selection rule.

I first use a broad parameter grid and then narrow it around the best value. Table 2 shows the three rounds. The selected model is finally refitted on all labeled observations and used for prediction.

Table 2: Task 3 hyperparameter exploitation rounds.

Data set	Model	Round	Search grid	Best parameter
A	Ridge	1	$z \in [1.5, 5.0]$	4.0
A	Ridge	2	$z \in [3.4, 4.6]$, step 0.2	3.8
A	Ridge	3	$z \in [3.65, 3.95]$, step 0.05	3.75
B	Lasso	1	$\alpha \in [0.10, 0.14]$, step 0.005	0.120
B	Lasso	2	$\alpha \in [0.116, 0.124]$, step 0.002	0.122
B	Lasso	3	$\alpha \in [0.1200, 0.1240]$, step 0.0005	0.1215
C	Lasso	1	$\alpha \in [0.15, 0.21]$, step 0.01	0.1900
C	Lasso	2	$\alpha \in [0.1800, 0.2000]$, step 0.0025	0.1875
C	Lasso	3	$\alpha \in [0.1850, 0.1900]$, step 0.0005	0.1865

Data set A. Data set A has 600 observations and 600 predictors. Ridge is the only model with positive validation R^2_{paper}. The best value is $z = 3.75$, with $R^2_{\text{paper}} = 0.042628$ and Sharpe 0.171803. The nearby values are similar, so further tuning is unlikely to help. Ridge is therefore used for the final prediction.

Data set B. Data set B has 240 observations and 2400 predictors. Lasso is suitable because it can remove many unhelpful predictors. The final choice is $\alpha = 0.1215$, with $R^2_{\text{paper}} = 0.001634$ and Sharpe 0.199729. Ridge and the neural network both have negative validation R^2, so Lasso is used for the final prediction.

Data set C. Data set C has 360 observations and 1800 predictors. The final Lasso value is $\alpha = 0.1865$, where $R^2_{\text{paper}} = 0.004597$ and the economic metrics are positive. A larger penalty of 0.30 sets every coefficient to zero, while Ridge and the neural network have clearly negative validation R^2. Lasso is therefore used for the final prediction.

3.2 Final models and prediction files

Table 3 gives the final choices. Each model is refitted on all labeled rows before predicting the public test features. The prediction files have the required row counts and column order.

Table 3: Task 3 final validation results selected by R^2_{paper}.

Data set	Model and parameter	R^2_{paper}	$\mathbb{E}[R^\pi]$	SR_{paper}	$\text{SR}_{\text{variance}}$	$\|\widehat{\beta}\|_2$
A	Ridge, $z = 3.75$	0.042628	0.056045	0.171803	0.174396	0.233292
B	Lasso, $\alpha = 0.1215$	0.001634	0.059076	0.199729	0.203836	0.320890
C	Lasso, $\alpha = 0.1865$	0.004597	0.003104	0.091762	0.092151	0.037342

Table 4: Task 3 final prediction files.

Data set	Observations	Prediction file
A	1000	`task3/A_ridge_result_20261010_155038/predictions_A.csv`
B	4000	`task3/B_lasso_result_20261010_155133/predictions_B.csv`
C	2000	`task3/C_lasso_result_20261010_155221/predictions_C.csv`

4 Task 4: Generalization and Overfitting in Neural Networks

4.1 Main finding of Montanari and Urbani

Montanari and Urbani [2] study a wide two-layer network,

$$f(x;\theta) = \frac{1}{m}\sum_{j=1}^{m} a_j \sigma(w_j' x), \tag{8}$$

on noisy data with a low-dimensional signal. Training has two main stages. First, the network learns the useful feature, so both training and test errors fall. Later, the second-layer weights grow and the network starts fitting noise. Training error continues to fall, but test error rises. The network can also move away from the true feature; the paper calls this *feature unlearning*. This explains why early stopping can improve test performance.

4.2 Finite-sample experiment

I test this idea with simulated returns. The predictors $x_t \in \mathbb{R}^{20}$ are standard Gaussian and

$$R_{t+1} = \sigma(u' x_t) + \varepsilon_{t+1}, \qquad \varepsilon_{t+1} \sim \mathcal{N}(0, \tau^2), \tag{9}$$

where u is a randomly drawn unit vector and

$$\sigma(z) = \sqrt{0.9}z + \frac{z^3 - 3z}{6}. \tag{10}$$

This follows the single-index setup in the paper's Figure 4. The network has $m = 32$ hidden units and $n = 192$ training observations. Validation and test sets contain 2000 and 5000 observations. I use $\tau \in \{0, 0.1, 0.2, 0.3, 0.4, 0.5\}$ and five replications.

The first-layer weights are normalized after every update, and all second-layer weights start at $a_j = 1$. SGD runs for 10,000 updates with batch size 100 and learning rate 0.1. The code is in `task4/generate_data.py` and `task4/task4.py`. At each checkpoint I record half-MSE, the weight-size measure

$$C(t) = \frac{1}{m}\sum_{j=1}^{m} |a_j(t)|, \tag{11}$$

and signal alignment

$$A(t) = \mathrm{Corr}^2(f(X_{\mathrm{te}}; \theta_t), \sigma(X_{\mathrm{te}} u)) \,. \tag{12}$$

The alignment measure shows how closely predictions follow the true signal. Metrics and figures are produced by `task4/test_metrics.py` and `task4/plot_results.py`.

4.3 Results

Figure 3 shows the main pattern. Early in training, training and test errors fall together and alignment rises above 0.9. With $\tau = 0$ or 0.1, test error keeps improving. From $\tau = 0.2$ onward, test error eventually rises, and higher noise moves the best stopping point earlier.

For $\tau = 0.3$, test half-MSE is lowest at update 2774 and rises by 10.1% by update 10,000. Complexity reaches 1.598, while alignment falls from 0.946 to 0.929. With $\tau = 0.5$, test error rises by 13.4% and alignment falls to 0.863. The noiseless case has no reversal. These results are consistent with the paper: more noise causes earlier and stronger overfitting.

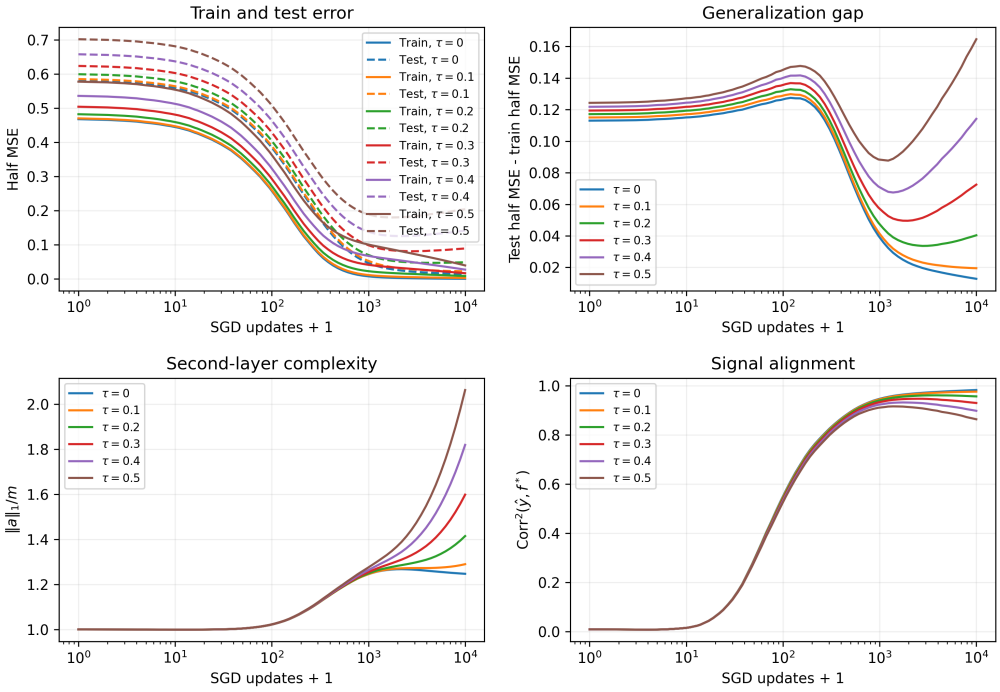


Figure 3: Training dynamics averaged over five replications. The panels show error, generalization gap, weight size, and signal alignment.

Table 5: Noise, early stopping, and late-training outcomes. The test-error increase compares update 10,000 with the minimum of the mean test curve.

τ	Best update	Best test half-MSE	Increase	Final $C(t)$	Final $A(t)$
0.0	10000	0.0133	0.0%	1.247	0.982
0.1	10000	0.0223	0.0%	1.290	0.975
0.2	4968	0.0473	3.6%	1.415	0.956
0.3	2774	0.0811	10.1%	1.598	0.929
0.4	2197	0.1255	12.7%	1.819	0.897
0.5	1740	0.1801	13.4%	2.063	0.863

4.4 Implication for financial machine learning

Here $u'x_t$ can be viewed as a return factor and τ as unpredictable return noise. The experiment shows that a lower training loss does not always mean a better forecasting model. For financial prediction, a chronological validation set should be used to choose the stopping point. Weight growth and prediction stability are also useful checks.

This is only a small simulation. It uses one network width, Gaussian data, and one stable factor. Real returns are less stable and include dependence, heavy tails, and trading costs. The experiment therefore supports the general idea of early stopping, but its numerical stopping points should not be used directly in a real strategy.

5 Task 5: Video and Reproducibility

The complete code is available at github.com/wanbic/DSA5205Project1. The repository includes the scripts, configuration files, requirements, and instructions for reproducing Tasks 1–4. A short video showing how to set up the environment and run the code will be uploaded to Canvas.

6 Conclusion

This project shows that model complexity is not always harmful in return prediction. Ridge reduces the instability near interpolation, while Lasso is less suitable for the dense signal in the simulation. For the three hidden data sets, validation selects Ridge for A and Lasso for B and C. The neural network experiment also shows that training loss can keep falling after test performance starts to worsen. Overall, regularization, validation, and early stopping are important for noisy financial data.

AI Use Statement

ChatGPT and OpenAI Codex were used to assist with various aspects, such as drafting and debugging code and editing the report.

References

[1] Kelly, B. T., Malamud, S., and Zhou, K. (2024). The virtue of complexity in return prediction. *Journal of Finance*, 79(1), 459–503.

[2] Montanari, A. and Urbani, P. (2025). Dynamical decoupling of generalization and overfitting in large two-layer networks. *Advances in Neural Information Processing Systems*, 38.